

Appendix

A – Transcript of Interview 1

Transcript was altered to preserve student anonymity

Client: Hey, I heard that computer science students have just started their IAs. Have you decided on anything yet?

Me: No, not really. I'm open to ideas though.

Client: Oh okay wait... Is it possible if you could make an app to, like, plan my day for me? I mean, perhaps it could tell me to do something at a certain time. I just find that making a calendar is so time consuming...

Me: What do you mean by time consuming, though?

Client: Um, like, I know I need to do some homework or revision sometime today, but I am always very undecided about when I should do what... I mean, I tried using a calendar before but everytime there is a new thing to be done, I have to readjust my schedule... and I'm very lazy haha...

Me: Oh... So is it that you don't like deciding when to do what and for how long when using a calendar?

Client: Yes! I mean, it just feels so troublesome... I mean, our school's online calendar is quite bad and I always manage to misclick and move some event around accidentally. So many times!

Me: Haha, yea I get what you mean. Have you tried other methods though? Like, using Apple Calendar or something...

Client: Honestly, they just seem all the same bro... I mean, I also tried those to-do applications but they all are so easy to forget about because they don't have an end time... Only a start time.

Me: Oh... So you want an automated time table system, right? Like, if you put something you want to do, then it tells you when to do it, is it?

Client: Yea... Kind of... But I also want to be able to do a faster way of adding my todo items... Like you know how when you Google search for something, you only have to type a few keywords?

Me: I guess... I think I know what you are talking about... Hold on, I need to ask something... When you add tasks or an event to your application or calendar or todo list, would you like the program to decide the start time of your task?

Client: Actually, that would be pretty cool. But, there will be definitely times when I want to set a time myself. Like, if I have tuition at a certain time, I don't want the app to be messing with that timing!

Me: Yea, definitely not. In that case, there would probably need to be an indication of a task that can be changed by the program and what cannot be changed... Hmmm, I think I'll go back home and think of something. I'll get back to you tomorrow.

Client: Ok! Thank you so much. Bye!

B – Transcript of interview 2

Transcript was altered to preserve student anonymity.

Client: Hi! How's the program thing going?

Me: Ah, I actually have a few questions about it... Um, so would it be correct to say that this app I'll be making is kind of like a calendar-todo list hybrid?

Client: Um... yea! That more or less sums it up.

Me: Okay, so, should this app be more towards a calendar or todo list? I mean, do you like to plan days ahead or is it just mainly short-term stuff you usually add to your todo list slash calendar?

Client: Definitely short-term, like within the current day. I'm way too lazy to plan far ahead, haha. Like, you know how calendars usually have a column for a day? And how events are shown as a 'block' in the column... I want to have that kind of look, but I don't need to see the whole week and I also don't want to have to use my mouse so much to click and drag here and there.

Me: Ok, ok, cool. So, like a to-do list view but with... the calendar look?

Client: Yeah, more or less.

Me: Right. Okay. Uh, how about repeating tasks? Would that be needed for the program?

Client: I mean, if you could, that would be nice. But, not necessary...

Me: Alright... Oh, quite importantly, should tasks be able to overlap with one another?

Client: Eh... Not necessarily, no... Yea, no. I don't see any point in doing that.

Me: Hmmm. Oh, remember you said that you wanted some of your added tasks to have a fixed timing?

Client: Yea, and others can be handled by the computer thing right? Like their start times?

Me: Yeah, what happens if you have two tasks with fixed timings and then you try and add a new task, but it can't fit between the two tasks?

Client: Um... Just put it somewhere else I guess.

Me: Right, but what if the task has to be split up? Like, fragmented?

Client: Oh... Um, that's an interesting idea. I mean, is it possible? If that occurs, then would your program be able to split it in a good way? Like, don't split a 30 minute task into many 5 minute tasks?

Me: It would be possible... Difficult, but possible. But I think we can discuss that at a later time... Just wanted to see if you were interested in the idea of tasks that can be split up automatically.

Client: Yea, it would be really cool if you could do something like that.

Me: Alright... Do you have any specific requests you want me to add?

Client: Umm, can you make it look nice? Like, the UI?

Me: I'll do my best, haha. Dark mode would be good?

Client: Oh yeah, definitely.

Me: Oh, and, just checking, what part of the calendar or the todo apps do you really find time consuming or troublesome? Like, specifically...

Client: Uh... Let me think... Oh, like, a lot of them want you to click on some button to add or remove an event, or if you want to edit the details of one, then a whole screen of options come up and it's such a hassle to edit the details...

Me: In that case, for the basic functions like adding, deleting and creating an event, what do you think of a command-style system?

Client: Oh, what do you mean by that?

Me: It's like, if you type out "add" for example, like how you would do in a search bar, and press enter, some input boxes will come up so you can enter the details of your

task... So, instead of having to click here and there to edit or change the details of a task, then you can just use your keyboard and do it really quickly.

Client: So would it give me suggestions? Like autocomplete? Also, there will be “edit” and “delete” other than “add” right?

Me: Yea! I mean, there will be phrases you can type to add, remove and delete tasks.

Client: Alright... how would I select different inputs though? After I press enter...

Me: You know when you press the tab key on some websites...

Client: Oh! You can do that? Like press tab and then it selects some other input box?

Me: Yea, exactly!

Client: Okay... Sounds good.

Me: Okay! Oh, one more thing, this program will be used on your laptop right?

Client: Yep. It's Windows 10, I think.

Me: Oh, ok. Sure, that works.

Client: Oh, just one more thing about the command bar thing... Do I need to type specific things like “add” or “delete” or can it have a vocabulary of things... Like “add” and “create” would give me the same inputs...

Me: Oh, yeah. That can be done.

Client: So it'll also have an autocomplete kind of thing right?

Me: Yea.

Client: Okay, I definitely want to see the final product when it's ready, haha. All the best!

Me: Thanks!

C – Summary of interview 3

This is a summary of the actual interview.

In the interview, the client and I agreed on the policy regarding how tasks would be split up. It was agreed that tasks will need to adhere to guidelines on whether or not it should be fragmented (split up into smaller tasks) as shown in the table below. Also, the maximum number of fragments for tasks that will be fragmented is outlined in the table.

Range of durations in minutes	Max no. of fragments allowed
duration $\leq$ 55	0
55 < duration $\leq$ 150	2
150 < duration $\leq$ 210	3
210 < duration	4

As the table indicates, a task with a duration of 55 minutes or less must never split up into smaller tasks even if it cannot be added with its current duration. In that case, the program will not add the task to the day. So, tasks that have a duration greater than or equal to 55 minutes would have their maximum number of fragments limited by their duration, as shown in the table.

It was agreed that the program will not always split tasks into their maximum allowed number of fragments. Instead, the program would always prioritise the least number of fragments possible in order for the task to be added to the day. Lastly, the client wished that the fragments were not to be split into tasks with a duration that was less than 30 minutes.

Importantly, the client and I agreed that there should not be fragments of fragments, as it could lead to confusion.

D – Summary of interview 4

This is a summary of the actual interview

After going over the behaviour of the fragmenting of Tasks, the client raised issues regarding the “unnatural” distribution of time when a Task was fragmented. For example, a 90-minute task could be split into one 42-minute fragment and a 48-minute fragment. The client clarified that she was more accustomed to Tasks that were multiples of 5 or 10, such as 35 minutes, 30 minutes, 60 minutes etc.

We agreed that the program would try and distribute Tasks accordingly, whenever possible. But, the program would still prioritise actually retaining time instead of removing time for a Task’s duration just to be a multiple of 5 or 10. So, some instances of fragments with “unnatural” durations would still be present, but not as frequently.

Apart from that, the client pointed out that she “really liked” the autocomplete feature of the program and thought the simplistic and keyboard-friendly design was “cool”.

E – Source Code

app/src/main/java/org/backend/Task.java

```
Java
package org.backend;

import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;

public class Task {

    private String name;

    private int duration; // in minutes

    private LocalDateTime timeStart;

    private Task fragmentOf;

    private boolean fixedTask;

    public Task(String title, int duration) {

        this.name = title;

        this.duration = duration;

        this.fragmentOf = null;

        this.fixedTask = false;

    }

    public Task(String title, int duration, LocalDateTime timeStart) {

        this.name = title;

        this.duration = duration;

        this.timeStart = timeStart;

        this.fixedTask = true;

    }

}
```



```

@Override public String toString() { // mainly for debugging purposes

    String type = (fixedTask) ? "[FIXED]" : "[AUTO]";

    return type + " " + name + " "

        + "    TIMING: " + timeStart.toString() + " to " +
getTimeEnd().toString();

}

public boolean isAfter(LocalTime time) {

    return timeStart.isAfter(time) && getTimeEnd().isAfter(time) ||
time.equals(timeStart);

}

public void bringForward(int minutes) {

    timeStart = timeStart.minusMinutes(minutes);

}

public int minsUntil(Task otherTask) {

    return (int) timeStart.plus(duration, ChronoUnit.MINUTES)

        .until(otherTask.getTimeStart(), ChronoUnit.MINUTES);

}

public boolean overlaps(Task otherTask) {

    int totalDuration = otherTask.getDuration() + duration;

    boolean occursBefore =

        minsUntil(otherTask) < 0 && Math.abs(minsUntil(otherTask)) >=
totalDuration;

    boolean occursAfter = minsUntil(otherTask) >= 0;

    return !(occursBefore ^ occursAfter);

```

```

    }

    public boolean isOngoing(LocalTime time) {
        return timeStart.isBefore(time) && getTimeEnd().isAfter(time);
    }

    public LocalTime getTimeEnd() {
        return timeStart.plus(duration, ChronoUnit.MINUTES);
    }

    public int getDuration() {
        return duration;
    }

    public LocalTime getTimeStart() {
        return timeStart;
    }

    public String getName() {
        return name;
    }

    public boolean isFixed() {
        return fixedTask;
    }

    public Task getFragmentOf() {
        return fragmentOf;
    }

```

```

    }

    public void setDuration(int duration) {
        this.duration = duration;
    }

    public void setTimeStart(LocalTime timeStart) {
        this.timeStart = timeStart;
    }

    public void setName(String title) {
        this.name = title;
    }

    public void setFragmentOf(Task fragmentOf) {
        this.fragmentOf = fragmentOf;
    }
}

```

app/src/main/java/org/backend/TaskHandler.java

```

Java
package org.backend;

import java.time.LocalTime;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.Arrays;

```

```

import java.util.Collections;

public class TaskHandler { // utility class

    public static ArrayList<Task> scheduledTasks;

    private TaskHandler() {}

    static {

        scheduledTasks = new ArrayList<>();

    }

    public static LocalTime getCurrentTime() {

        return LocalTime.now().truncatedTo(ChronoUnit.MINUTES);

    }

    /**
     * Adds a <code>FixedTask</code> to the <code>scheduledTasks</code>
     * ArrayList. Returns ints depending on the method outcome.
     *
     * @param task the <code>FixedTask</code> that should be added
     * @return <b>-2</b> a <code>task</code> with the same name already exists
     * <li><b>-1</b> the provided <code>task</code> has an incorrect duration,
     * such as if it was
     *
     * <b>0</b>
     *
     * <li><b>0</b> the provided <code>task</code> was successfully added
     * <li><b>1</b> there is not enough free time for the <code>task</code> to
     * be added <li><b>2</b> the provided <code>task</code> is conflicting with
     * an existing scheduled task

```

```

    */

    private static int addFixedTask(Task task) {

        final LocalTime CURRENT_TIME = getCurrentTime();

        // error checking

        if (task.getDuration() == 0)

            return -1;

        if (taskAlreadyExist(task))

            return -2;

        // a task to midnight (24:00) will be auto corrected to 23:59

        if (task.getTimeStart().until(task.getTimeEnd(), ChronoUnit.HOURS)

            == -1 * task.getTimeStart().getHour())

            task.setDuration(task.getDuration() - 1);

        // prevent task from going to next day

        else if (task.getTimeStart().isAfter(task.getTimeEnd()))

            return 1;

        if (scheduledTasks.isEmpty()) { // if no existing tasks, just add the
task

            scheduledTasks.add(task);

            return 0;

        }

        int totalFreeTime = 0;

        ArrayList<Integer> freeTimes = getFreeTime(CURRENT_TIME);

        for (Integer integer : freeTimes) // gets total free time in day

            totalFreeTime += integer.intValue();

        // checks to make sure there is enough free time

```

```

    if (task.getDuration() > totalFreeTime)

        return 1;

    int pos = 0;

    for (int i = 0; i < scheduledTasks.size(); i++) {

        Task current = scheduledTasks.get(i);

        if (task.overlaps(current)) // tasks must not overlap

            return 2;

        if (current.getTimeStart().isBefore(task.getTimeStart()))

            pos++; // finds position in scheduledTasks to add task to

    }

    scheduledTasks.add(pos, task); // adds task

    return 0; // success code
}

/**
 * Gets every duration of uninterrupted time in which there are no ongoing
 * tasks. In other words, it gets the durations in between tasks. All free
 * time in the past is ignored. It is retrieved in an
 * <code>ArrayList</code> of <code>Integers</code>, with durations in
 * minutes. The returned <code>ArrayList</code> is ordered relative to the
 * order of existing tasks in the day.
 *
 * @param currentTime the current time according to the user's system

```

```

    * @return <b>ArrayList</b> collection of free time in minutes
    */

    private static ArrayList<Integer> getFreeTime(LocalTime currentTime) {

        if (scheduledTasks.isEmpty())

            return null;

        ArrayList<Integer> freeTimes = new ArrayList<>();

        Task nextTask = getNextOccuringTask(currentTime);

        int posOfNext = scheduledTasks.indexOf(nextTask);

        int minsFromNowToNext = Integer.MAX_VALUE;

        int minsFromPrevToNext = Integer.MAX_VALUE;

        minsFromNowToNext = (nextTask == null) // adds the free time at the
start
        ? (int) currentTime.until(LocalTime.MAX, ChronoUnit.MINUTES) // if
no next task
        : (int) currentTime.until(nextTask.getTimeStart(),
ChronoUnit.MINUTES); // if got next

        // if on-going task may end closer to start of nextTask compared to
current time

        Task currentTask = getCurrentOccuringTask(currentTime);

        if (currentTask != null && nextTask != null)

            minsFromPrevToNext = (int) currentTask.minsUntil(nextTask);

        int minsFromNext = Math.min(minsFromNowToNext, minsFromPrevToNext); //
gets the first

```

```

//
    occurring free time

    freeTimes.add(minsFromNext);

    if (nextTask == null)
        return freeTimes; // returns duration from current time to 23:59

    // adds the free times found between existing tasks
    for (int i = posOfNext; i < scheduledTasks.size() - 1; i++) {
        Task current = scheduledTasks.get(i);
        Task next = scheduledTasks.get(i + 1);
        freeTimes.add(current.minsUntil(next));
    }

    // adds the free time from the end of last task to 23:59
    Task lastTask = scheduledTasks.getLast();
    freeTimes.add((int) lastTask.getTimeEnd().until(LocalTime.MAX,
ChronoUnit.MINUTES));

    return freeTimes;
}

private static Task getCurrentOccuringTask(LocalTime currentTime) {
    for (Task task : scheduledTasks) {
        if (task.isOngoing(currentTime))
            return task;
    }
    return null;
}

```



```

    }

    /**
     * Returns the next task that occurs from the present time. If there is an
     * ongoing task, it will be ignored. An ongoing task is classified as a
task
     * with a start time before the current time, and an end time after the
     * current time.
     *
     * @param currentTime the current time of the user
     * @return the next Task that has not yet started
     */
    public static Task getNextOccuringTask(LocalTime currentTime) {
        if (scheduledTasks.isEmpty())
            return null;

        int pastPos = 0;
        int futurePos = scheduledTasks.size() - 1;
        // binary search for the next task
        while (pastPos < futurePos) {
            int currentPos = (pastPos + futurePos) / 2;
            Task task = scheduledTasks.get(currentPos);
            if (!task.isAfter(currentTime)) {
                pastPos = currentPos + 1; // immediate next. not on-going task
            } else {
                futurePos = currentPos;
            }
        }
    }
}

```

```

        // if there is no next task, return null

        Task nextTask = scheduledTasks.get(pastPos);

        if (!nextTask.isAfter(currentTime)) {
            nextTask = null;
        }

        return nextTask;
    }

    /**
     * Attempts to automatically allocate tasks as early as possible, relative
     * to the user's system time. It is able to add tasks in between other
     * existing tasks. Returns integers, based on the outcome of the
     * allocation.
     *
     * @param task the <code>task</code> to be automatically allocated
     * @return <b>-2</b> a <code>task</code> with the same name already exists
     * <li><b>-1</b> the <code>task</code> in parameter has an unprocessable
     * duration. For example, if the <code>task</code> duration is 0.
     * <li><b>0</b> successful allocation of <code>task</code> provided
     * <li><b>1</b> there is not enough free time to allocate the
     * <code>task</code>
     */
    private static int autoAllocate(Task task) {
        final LocalTime CURRENT_TIME = getCurrentTime();

        if (task.getDuration() == 0)
            return -1;
    }

```

```

if (task.getFragmentOf() == null && taskAlreadyExist(task))
    return -2;

// get all free times
ArrayList<Integer> freeTimes = getFreeTime(CURRENT_TIME);

int nextTaskPos =
scheduledTasks.indexOf(getNextOccuringTask(CURRENT_TIME));

int taskDuration = task.getDuration();

// if scheduledTasks is empty or if there is no next task
if (nextTaskPos < 0
    && CURRENT_TIME.until(LocalTime.MAX, ChronoUnit.MINUTES) >=
taskDuration) {
    task.setTimeStart(CURRENT_TIME);
    scheduledTasks.add(task);
    return 0;
} else if (nextTaskPos < 0) // if no next task but too close to end of
day
    return 1;

// allocating tasks in between existing tasks. think of each free time
// as being found in front of an existing task. this is why the last
// free time has to be handled separately (delete me and put in
explanation)

// allocate tasks in between tasks
for (int i = nextTaskPos; i < scheduledTasks.size(); i++) {

```

```

        int freeTime = freeTimes.get(i - nextTaskPos).intValue();

        if (freeTime >= taskDuration) {

            task.setTimeStart(

                scheduledTasks.get(i).getTimeStart().minus(freeTime,
ChronoUnit.MINUTES));

            scheduledTasks.add(i, task);

            return 0;

        }

    }

    // allocating a task at the end of the day
    int freeTime = freeTimes.getLast().intValue();
    if (freeTime >= taskDuration) {

        task.setTimeStart(scheduledTasks.getLast().getTimeEnd());

        scheduledTasks.add(task);

        return 0;

    }

    // if taskDuration is too big for any free time
    if (task.getFragmentOf() != null) // do not allow frags of frags

        return 1;

    // sorts in descending order
    Collections.sort(freeTimes, Collections.reverseOrder());

    int[] fragFormat = fragsFormatter(taskDuration, freeTimes); // get
frags duration

    if (fragFormat.length == 0) { // not enough free time / task is too
small to be frag

```

```

        return -1;
    }

    for (int i = 0; i < fragFormat.length; i++) {
        Task fragTask = new Task(task.getName(), fragFormat[i]);
        fragTask.setFragmentOf(task);
        autoAllocate(fragTask);
    }

    return 0;
}

/**
 * This method first assigns a maximum number of times a task can be split
 * from 2 to 4 depending on the duration of the task. It tries to apply
 * smaller fragmentations than the maximum to prevent unnecessary
 * splitting.
 *
 * @param taskDuration
 * @param freeTimes
 * @return returns an array of integers that dictate how the original task
 *         will be split up. An
 *         array of length 0 indicates that the original task should not be
 *         fragmented.
 */
private static int[] fragsFormatter(int taskDuration, ArrayList<Integer>
freeTimes) {
    if (taskDuration <= 55)
        return new int[0];
}

```

```

int maxFrag = -1;

// client defined these constants to decide max no. of frags for
// certain durations

final int[] TIME_LIMITS = {55, 150, 210};

final int[] MAX_FRAGS = {2, 3, 4};

for (int i = 0; i < TIME_LIMITS.length - 1; i++) {
    if (maxFrag == -1 && TIME_LIMITS[i] < taskDuration
        && taskDuration <= TIME_LIMITS[i + 1])
        maxFrag = MAX_FRAGS[i];
}

if (maxFrag == -1)
    maxFrag = MAX_FRAGS[MAX_FRAGS.length - 1];

// get minimum no. of possible frags

int duration = taskDuration;

int minFrag = 0;

while (duration > 0 && minFrag < freeTimes.size()) {
    duration -= freeTimes.get(minFrag++).intValue();
}

if (duration > 0 || minFrag > maxFrag) // not enough time in each
free time

    return new int[0];

int[] fragTemplate = new int[minFrag];

final int MIN_FRAG_DURATION = 30; // client defined this

duration = taskDuration; // reset the duration

```

```

// populate fragTemplate greedily

int freeTimePos = 0;

    for (int i = 0; i < fragTemplate.length && freeTimePos <
freeTimes.size(); i++) {

        int freeTime = freeTimes.get(freeTimePos++).intValue();

        if (freeTime >= MIN_FRAG_DURATION && duration > freeTime) {

            fragTemplate[i] = freeTime;

            duration -= freeTime;

        } else if (freeTime >= MIN_FRAG_DURATION) {

            fragTemplate[i] = duration;

            duration = 0;

        } else {

            --i;

        }

    }

    if (duration == 0)

        return fragTemplate;

// attempt to balance out fragTemplate values

    for (int i = minFrag; i <= maxFrag; i++) { // from min frags to max
frags allowed

        ArrayList<Integer> modFreeTimes = new ArrayList<>();

        // make a shallow copy of free

                                freeTimes.forEach((d)    ->
modFreeTimes.add(Integer.valueOf(d.intValue())));

        // make all durations a multiple of 5 or 10

        fragTemplate = makeFiveTenMultiple(i, taskDuration);

```

```

        int templatePos = 0;

        for (int j = 0; j < modFreeTimes.size() && templatePos <
fragTemplate.length; j++) {

            // get free time at j

            int currentFree = modFreeTimes.get(j).intValue();

            if (currentFree >= fragTemplate[templatePos]) { // if free time
more than frag time

                // occupy free time and repeat without

                // incrementing j. increment templatePos

                modFreeTimes.set(

                                                j--, Integer.valueOf(currentFree -
fragTemplate[templatePos++]));

            }

        }

        if (templatePos == fragTemplate.length)

            return fragTemplate;

    }

    // if all methods of task fragmentation fails

    return new int[0];

}

/**
 * Creates and returns an integer array with a specified length,
 * <code>arrLen</code> with values at position 1 and greater being

```



```

    * multiples of 5 or 10 and having all its values in the array sum up to a
    * specified integer, <code>sum</code>. The value at position 0 can still
    * be a multiple of 5 or 10. However, it is not guaranteed.
    *
    * @param arrLen length of array to create
    * @param sum     the sum of all elements in the new array
    * @return <b>int[]</b> array of integers that are all definitely multiples
    *         of 5 or 10, except for the integer at position 0.
    */
private static int[] makeFiveTenMultiple(int arrLen, int sum) {
    int[] returnArray = new int[arrLen]; // array to return

    // if array length is even, values of array strictly after pos 0 will
    // be multiples of 10. else, 5

    final int MULTIPLE = (arrLen % 2 == 0) ? 10 : 5;

    int remainder = sum % MULTIPLE; // get the remainder

    // fills array with multiples of 5 or 10

    Arrays.fill(returnArray, (sum - remainder) / arrLen);

    returnArray[0] += remainder; // adds remainder to value at pos 0

    return returnArray;
}

public static int addTask(Task taskToAdd) { // wrapper method

    return (taskToAdd.isFixed()) ? addFixedTask(taskToAdd) :
autoAllocate(taskToAdd);
}

/**

```

```

    * Finds a task based on a provided name. Only returns an exact match.
    *
    * @param taskName name of task to return
    * @return <b>Task</b> with the exact matching name
    */
    public static Task findTask(String taskName) {
        for (Task task : scheduledTasks)
            if (task.getName().equals(taskName))
                return task;

        return null;
    }

    /**
     * Checks if a task with the same name already exists.
     *
     * @param task task with a name
     * @return <b>boolean</b> <code>true</code> if a task with the same name as
     *         the provided task
     *         exists
     */
    private static boolean taskAlreadyExist(Task task) {
        String taskName = task.getName();

        for (Task t : scheduledTasks) {
            String currentTaskName = t.getName();

            if (taskName.equals(currentTaskName))
                return true;
        }

        return false;
    }

```

```

}

/**
 * Removes an existing task. If the task is a fragment, its other fragments
 * will also be deleted.
 *
 * @param unTask erased from the past and present. the task... never
 *     existed
 */
public static void removeTask(Task unTask) {
    scheduledTasks.removeIf((task)
        -> task.equals(unTask)
        || task.getFragmentOf() != null && unTask.getFragmentOf() !=
null
        && task.getFragmentOf().equals(unTask.getFragmentOf()));
}

/**
 * Replaces an old Task with a new Task.
 *
 * @param oldTask the Task to be replaced
 * @param newTask the Task that will replace
 *     oldTask
 * @return int success codes of {@link #addFixedTask(FixedTask)}
 */
public static int replaceTask(Task oldTask, Task newTask) {
    ArrayList<Task> backupScheduledTasks = new ArrayList<>();

```

```

        for (Task task : scheduledTasks) // soft copy scheduledTasks
            backupScheduledTasks.add(task); // into a backup collection

        removeTask(oldTask); // removes the old task from the schedule

        int successCode = addTask(newTask); // add new task

        if (successCode != 0) // if failure, restore from backup
            scheduledTasks = backupScheduledTasks;

        return successCode; // return success code
    }
}

```

app/src/main/java/org/backend/Rosetta.java

```

Java
package org.backend;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import javafx.scene.Scene;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.scene.web.WebView;
import netscape.javascript.JSObject;
import org.backend.commands.Command;

```

```

// singleton class

public class Rosetta { // allows communication between Java and JavaScript

    private static Rosetta instance = null;

    private WebView wv = null;

    private Command[] cmds = null;

    private int selectedCmdPos = 0;

    private Rosetta(WebView wv) {

        this.wv = wv;

    }

    public static void initialise(WebView wv) {

        if (instance == null)

            instance = new Rosetta(wv);

        wv.getEngine().documentProperty().addListener((newDoc) -> { // prevents
garbage collection

            if (newDoc != null) { // on document load

                instance.enableJavaScriptComms();

                instance.populateTasks();

            }

        });

    }

    public static Rosetta getCurrentInstance() {

        return instance;

    }
}

```

```

/**
 * Monitors for hotkeys such as Enter, Tab, Space and Escape so that they
can be forwarded to
 * the frontend.
 *
 * @param oceania the main javafx scene (the app window)
 */
public static void startBigBrother(Scene oceania) {
    // alt. solution as js keydown, keypress and keyup events dont work
here
    Rosetta rosetta = Rosetta.getCurrentInstance();
    if (rosetta == null)
        return;
    /*
        * ChatGPT was used to generate the proper syntax for
"oceania.addEventFilter
        * (KeyEvent.Key_PRESSED, event -> {})" Code within the lambda function
was NOT generated,
        * in any way, by AI
    */
    oceania.addEventFilter(KeyEvent.KEY_PRESSED, event -> {
        String selectedIndexParam =
String.valueOf(rosetta.getSelectedCmdPos());
        String key = "";
        KeyCode keyCode = event.getCode();

        switch (keyCode) { // converts recognised hotkeys into string

```

```

        case KeyCode.ENTER:
            key = "Enter";
            break;

        case KeyCode.TAB:
            key = "Tab";
            break;

        case KeyCode.SPACE:
            key = "Space";
            break;

        case KeyCode.ESCAPE:
            key = "Escape";
            break;

        default:
            break;
    }

    if (!key.isEmpty()) { // sends detected key to the front end
        rosetta.executeJS("handleCommandSubmitKey", key,
selectedIndexParam);
        event.consume();
    }
});
}

public void enableJavaScriptComms() {
    JSObject window = (JSObject) ww.getEngine().executeScript("window");
    window.setMember("rosetta", instance); // this (instance) is exposed to
javascript frontend

```

```

    }

    public void populateTasks() { // adds tasks to the front end

        ArrayList<Task> tasks = TaskHandler.scheduledTasks;

        for (Task task : tasks) {

            String taskTitle = task.getName();

            String taskTimeStart = task.getTimeStart().toString();

            String taskTimeEnd = task.getTimeEnd().toString();

            String fixedStatus = (task.isFixed()) ? "manual" :
"auto-allocated";

            executeJS("addTask", taskTitle, taskTimeStart, taskTimeEnd,
fixedStatus);

        }

    }

    public void populateArgs() { // calls frontend methods to create input
boxes

        executeJS("initArgsContainer"); // starts a container to hold the
inputs

        String[] argTypes = cmds[selectedCmdPos].getArgTypes(); // gets arg
types

        for (int i = 0; i < argTypes.length; i++) {

            executeJS("createArg", argTypes[i], String.valueOf(i));

        }

        executeJS("updateSelectedArg");

```



```

    }

    /**
     * Called when the user types in the command bar. Calls JavaScript methods
    to display frontend
     * information.
     *
     * @param input the user input
     */
    public void handleCommandInput(String input) {
        executeJS("adjustInputWidth"); // frontend ui adjustment to fit text
        being typed

        Command[] commands = Interpreter.getPotentialCommands(input); // gets
        best matching cmds for

                                                                    // input

        Collections.reverse(Arrays.asList(commands)); // the best match will be
        displayed at the

                                                                    // bottom of the list

        cmds = commands.clone(); // store a copy of the cmds that are being
        displayed to the user

        selectedCmdPos = cmds.length - 1; // updates the value of the current
        selected cmd position

        executeJS("prepareCommandView"); // adds a (list) container for the
        commands to the UI
    }

```

```

        for (int i = 0; i < commands.length; i++) {
            String cmdName = commands[i].getKeywords()[0];
            String cmdDesc = commands[i].getDescription();

            executeJS("addCommand", cmdName, cmdDesc); // command is added and
displayed
        }

        executeJS("selectCommandAtPos",
            String.valueOf(selectedCmdPos)); // indicate current selected cmd
to user
    }

    public void handleCommandunfocus() { // called from index.html
        // deletes list of cmds in the frontend when the cmd bar is not
selected
        executeJS("deleteCommandView");
    }

    /**
     * Called by the JavaScript frontend when the user presses enter to execute
a command after
     * setting input values. It cleans up frontend ui elements and advises the
user if there are
     * required inputs that are left blank.
     *
     * @param jsObject Javascript array of input values as strings
     */

```

```

public void executeCommand(Object jsObject) {

    String[] args = jsToJavaStringArrayConvert(jsObject);

    if (args == null)

        return;

    boolean[] valids = cmds[selectedCmdPos].execute(args); // executes the
command

    // cleans up the user interface

    if (Interpreter.boolArrayAllTrue(valids)) {

        executeJS("resetCommandView");

        executeJS("clearTaskList");

        executeJS("resetArgInputs");

        populateTasks(); // refreshes the task list

        return;

    }

    // indicate any errors to the user

    boolean[] argOptionals = cmds[selectedCmdPos].getArgOptionals();

    for (int i = 0; i < valids.length; i++) {

        if (valids[i])

            executeJS("setArgAsNormal", String.valueOf(i));

        else {

            executeJS("setArgAsInvalid", String.valueOf(i));

        }

        if (!argOptionals[i] && args[i].isBlank())

            displayError("Please enter required input(s)");

    }
}

```

```

    }

    /**
     * Display the argument hint at a certain index of the currently selected
command to the
     * frontend.
     *
     * @param index the index of the argument
     */
    public void requestArgHint(int index) {
        Command selCommand = cmds[selectedCmdPos];
        String argTitle = selCommand.getKeywords()[0];
        String argBody = selCommand.getArgHints()[index];
        executeJS("showArgHint", argTitle, argBody);
    }

    public void displayError(String msg) { // exposes the javascript method to
other classes
        executeJS("showError", msg);
    }

    public int getSelectedCmdPos() {
        return selectedCmdPos;
    }

    public void setSelectedCmdPos(int selectedCmdPos) {
        this.selectedCmdPos = selectedCmdPos;
    }

```

```

/**
 * Converts a string Javascript array to a Java <code>String</code> array
 *
 * @param jsObject the Javascript string array to convert
 * @return <b>String[]</b> the converted <code>String</code> array
 */

```

```

private String[] jsToJavaStringArrayConvert(Object jsObject) {
    // sourced from chatgpt
    if (jsObject instanceof JSObject jsStringArray) {
        int len = (int) jsStringArray.getMember("length");
        String[] javaArray = new String[len];
        // copies string values to new array
        for (int i = 0; i < len; i++) {
            javaArray[i] = (String) jsStringArray.getSlot(i);
        }

        return javaArray;
    }

    return null;
}

```

```

/**
 * This method executes a javascript method from java. Do not call if
methodName is from a
 * userInput as it is unsanitised. args are sanitised. Only string args are
allowed.

```

```

*
* @param methodName name of the method, without the () at the end
* @param args any arguments that should be included
*/

private void executeJS(String methodName, String... args) {
    StringBuilder methodCall = new StringBuilder(methodName);
    methodCall.append("("); // open bracket to start adding args

    for (int i = 0; i < args.length; i++) {
        methodCall.append("String.raw`" + args[i] + "`"); // appends args
        // and sanitises

        if (i < args.length - 1)
            methodCall.append(", "); // specify that there is next arg
    }

    methodCall.append(");"); // close bracket to complete the method call
    ww.getEngine().executeScript(methodCall.toString()); // executes
    // javascript
}
}

```

app/src/main/java/org/backend/Interpreter.java

```

Java
package org.backend;

import java.time.LocalDateTime;

```

```

import org.backend.commands.AddCMD;

import org.backend.commands.Command;

import org.backend.commands.ModifyCMD;

import org.backend.commands.RemoveCMD;


public class Interpreter { // utility class


    private Interpreter() {}


    /**
     * Checks if a boolean array contains values that are all true.
     *
     * @param bools
     * @return returns true if there are no false elements in bools, false if
there is at least one
     *      false element.
     */
    public static boolean boolArrayAllTrue(boolean[] bools) {
        for (boolean b : bools) {
            if (!b)
                return false;
        }
        return true;
    }


    /**
     * This method attempts to parse common short-hand expressions of time,
such as "12pm" and

```

* "2am". It converts these shortened time expressions from String to a new `LocalTime` object.

*

* @param `rawStringTime` the `String` to be converted into a `LocalTime` object

* @return a new `LocalTime` object

* **if input could not be parsed**

*/

```
public static LocalTime parseTime(String rawStringTime) {
    if (rawStringTime.isBlank())
        return null;

    String stringInput = rawStringTime.toLowerCase();
    boolean isPM = stringInput.contains("pm");
    boolean isAM = stringInput.contains("am");
    StringBuilder numerics = new StringBuilder();
    char[] chars = stringInput.toCharArray();
    // copy all digits to numerics
    for (char c : chars) {
        if (Character.isDigit(c))
            numerics.append(c);
    }
    // make numerics length of 4
    if (numerics.length() == 1) {
        numerics.insert(0, "0");
        numerics.append("00");
    } else if (numerics.length() == 3) {
        numerics.insert(0, "0");
    }
}
```



```

    } else if (numerics.length() == 2) {
        numerics.append("00");
    }

    if (numerics.length() != 4)
        return null;

    // get hours
    int hour = Integer.parseInt(numerics.substring(0, 2));
    int minute = Integer.parseInt(numerics.substring(2));

    // conversions to 24 hour format
    if (hour != 12 && isPM)
        hour += 12;
    else if (hour == 12 && isAM) {
        hour = 0;
    }

    return LocalTime.of(hour, minute);
}

/**
 * Converts a string format of a duration to minutes as an int. For
 * example, 2h 30m returns 150
 *
 * (minutes). 2 hours and 30 minutes also returns 150 (minutes).
 *
 * @param stringDuration input <code>String</code> to convert into a
 * duration
 *
 * @return <b>duration in minutes</b> as an <code>int</code> value greater
 * than 0

```

```

*      <li><b>-1</b> if the parsed duration is less than or equal to 0
*/

public static int parseDuration(String stringDuration) {
    stringDuration = stringDuration.toLowerCase();

    char[] characters = stringDuration.toCharArray();

    int minutes = 0;
    int hours = 0;

    StringBuilder currentNum = new StringBuilder();

    for (char c : characters) { // extract digit
        if (Character.isDigit(c)) {
            currentNum.append(c);
        } else if (c == 'h') { // identify if currentNum is in hours
            hours = Integer.parseInt(currentNum.toString());
            currentNum = new StringBuilder();
        } else if (c == 'm') { // identify if currentNum is in minutes
            minutes = Integer.parseInt(currentNum.toString());
            currentNum = new StringBuilder();
        }
    }

    if (minutes + hours <= 0) // if no amount of time is collected
        return -1;

    return minutes + 60 * hours; // return duration in minutes
}

/**

```

```

        * Retrieves the best three commands as a sorted array of
<code>Command</code>objects, starting
        * with the closest matches, for a given <code>userInput</code>. If there
is an exact match
        * between the <code>userInput</code> and a <code>Command</code> object, an
array consisting of
        * that single <code>Command</code> will be returned.
        *
        * @param userInput the command that is being typed out by the user
        * @return an <b>array</b> of <code>Command</code> elements
        */
public static Command[] getPotentialCommands(String userInput) {
    Command[] commands = {new AddCMD(), new ModifyCMD(), new RemoveCMD()};
    Command[] potentialCommands = new Command[3];

    if (userInput.length() == 0) { // if the user input is empty, just
return first three

                                // without processing
        for (int i = 0; i < potentialCommands.length; i++)
potentialCommands[i] = commands[i];

        return potentialCommands;
    }

    // selection sort. closest matching cmd is at the front
    for (int i = 0; i < commands.length; i++) {
        int shortestDistPos = 0;
        int shortestDis = Integer.MAX_VALUE;

```

```

        for (int j = i; j < commands.length; j++) {
            Command cmd = commands[j];

            int dist = matchCommandNames(userInput, cmd);

            if (dist < shortestDis) {
                shortestDis = dist;
                shortestDistPos = j;
            }
        }

        // swap
        Command temp = commands[i];
        commands[i] = commands[shortestDistPos];
        commands[shortestDistPos] = temp;
    }

    if (commands[0].getKeywords()[0].equals(userInput)) { // if there is an
exact match

        potentialCommands = new Command[1];
        potentialCommands[0] = commands[0];

        return potentialCommands; // return the exact match only
    }

    for (int i = 0; i < potentialCommands.length; i++) // else, get best 3
cmds

        potentialCommands[i] = commands[i];

```

```

        return potentialCommands; // return best 3 cmds
    }

    /**
     * Adjusts the given <code>Command</code> to better match the user input.
     Since commands accept
     * multiple different keywords, such as "create" and "add" for adding a new
     task to the day,
     * this method will allow the user interface to display the best matching
     keyword for a given
     * <code>String</code> and <code>Command</code>.
     *
     * @param input the user input
     * @param cmd the command that should try and match the user input
     * @return <b>int</b> value of the levenshtein distance between the best
     matching <code>cmd
     *
     * </code> keyword to the <code>input</code>
     */
    private static int matchCommandNames(String input, Command cmd) {
        String[] names = cmd.getKeywords();
        int bestDist = Integer.MAX_VALUE;
        int posOfBestDist = -1;

        // get the levenshtein distances of each cmd keyword
        for (int i = 0; i < names.length; i++) {
            int currentDist = getLevenshteinDistance(input.toLowerCase(),
names[i].toLowerCase());

```

```

        if (currentDist < bestDist) { // store best distance

            bestDist = currentDist;

            posOfBestDist = i;

        }

    }

    // swap best matching keyword with default keyword

    String temp = names[posOfBestDist];

    names[posOfBestDist] = names[0];

    names[0] = temp;

    return bestDist; // returns best distance for the provided cmd
}

// method below adapted from
//
https://www.geeksforgeeks.org/java-program-to-implement-levenshtein-distance-computing-algorithm/
/**
 * Edit distance algorithm which measures how closely two strings match.
The smaller the
 * distance, the closer the match of the two provided strings. Method has a
bias, where having
 * to append characters for a match of the two strings is penalised less
than replacement or
 * removal since this method is often called when the user is in the
process of typing
 * something.

```

```

*
* @param input first <code>String</code>
* @param target second <code>String</code>
* @return <b>int</b> the levenshtein distance distance between the two
strings provided in the
*     parameters
*/
public static int getLevenshteinDistance(String input, String target) {
    int[][] dp = new int[input.length() + 1][target.length() + 1];

    for (int i = 0; i < dp.length; i++) // scenario where second str is
empty
        dp[i][0] = i;

    for (int i = 0; i < dp[0].length; i++) // scenario where first str is
empty
        dp[0][i] = i;

    for (int i = 1; i < dp.length; i++) {
        for (int j = 1; j < dp[i].length; j++) {
            int append = dp[i][j - 1] + 1; // append to input
            int remove = dp[i - 1][j] + 2; // remove from input (double
penalty)
            int replace = dp[i - 1][j - 1]; // assume no replacement

            if (input.charAt(i - 1) != target.charAt(j - 1)) // if chars at
pos i dont match
                replace += 2; // add replacement penalty (double penalty)

```

```

        dp[i][j] = Math.min(append,
            Math.min(remove,
                replace)); // find min of append, remove and replace
ops
    }
}

return dp[input.length()][target.length()]; // returns the last element
of the 2d array

// (bottom right element)
}
}

```

app/src/main/java/org/backend/Commands/AddCMD.java

```

Java
package org.backend.commands;

import java.time.LocalDateTime;
import java.util.Arrays;
import org.backend.Interpreter;
import org.backend.Rosetta;
import org.backend.Task;
import org.backend.TaskHandler;

public class AddCMD extends Command {
    public AddCMD() {

```



```

    super();

    setKeywords("add", "create", "new", "make");

    setDescription("Adds a new task to the day");

    setArgTypes("Text", "Duration", "Time");

    setArgHints("Set task title", "Set task duration", "(Optional) Set task
time");

    setArgOptionals(false, false, true);
}

@Override public boolean[] execute(String[] stringArgs) { // adds a new
task to the day

    boolean[] argValids = verifyArgs(stringArgs);

    if (!Interpreter.boolArrayAllTrue(argValids))

        return argValids;

    // handle first arg

    String name = null;

    if (!stringArgs[0].isBlank())

        name = stringArgs[0];

    else

        argValids[0] = false;

    // handle second arg

    int durationInMins = Interpreter.parseDuration(stringArgs[1]);

    if (durationInMins <= 0)

        argValids[1] = false;

    // handle third arg

    LocalTime parsedTime = Interpreter.parseTime(stringArgs[2]);

    if (parsedTime == null && !stringArgs[2].isBlank())

        argValids[2] = false;

```

```

Rosetta rosetta = Rosetta.getCurrentInstance();

if (!Interpreter.boolArrayAllTrue(argValid))
    return argValid;

// execute
Task newTask = (stringArgs[2].isBlank()) ? new Task(name,
durationInMins)
: new Task(name,
durationInMins, parsedTime);

int statusCode = TaskHandler.addTask(newTask);

// error handling
switch (statusCode) {
    case -2:
        rosetta.displayError("A task with the same name already
exists.");
        argValid[0] = false;
        break;
    case -1:
        rosetta.displayError("Could not fragment task! Check your input
value.");
        argValid[1] = false;
        break;
    case 0: // do nothing and be happy
        break;
    case 1:

```

```

        rosetta.displayError("Insufficient free time to add the
specificied task.");
        argValids[1] = false;
        break;
    case 2:
        rosetta.displayError("You cannot add a task that overlaps
existing tasks.");
        argValids[2] = false;
        break;
    default:
        rosetta.displayError(
            "An error occurred. Code: " + statusCode + ". Contact the
developer.");
        Arrays.fill(argValids, false);
        break;
    }

    return argValids; // bool array with all true values means command was
successfully executed
}
}

```

app/src/main/java/org/backend/Commands/Command.java

```

Java
package org.backend.commands;

import java.util.Arrays;

```

```

public abstract class Command { // command object should not be created. only
objects that inherit

                                // command

    private String[] keywords; // first element is default keyword. all others
are alternate

    private String description; // description of command

    private String[] argTypes; // argument names (ui)

    private String[] argHints; // argument hints (ui)

    private boolean[] argOptionals; // indicates which arguments are optional.
true if optional.

    public Command() {}

    public abstract boolean[] execute(String[] stringArgs);

    /**
        * Before carrying out command execution, this method should be run to
ensure the provided

        * arguments are processable by commands. Command execution should only be
done if this method

        * returns a boolean array of all true values.
        *
        * @param args user input

        * @return array of boolean values. false values indicate an invalid
argument

        */

    protected boolean[] verifyArgs(String[] args) {

```

```

        boolean[] valids = new boolean[getArgOptionals().length];

        Arrays.fill(valids, true); // assume everything is valid first

        boolean[] argOptionals = getArgOptionals(); // indicates which args are
optional

        for (int i = 0; i < args.length; i++) { // for all strings

            char[] characters = args[i].toCharArray(); // convert input to
chars

            valids[i] = false; // assume invalid

            for (char c : characters) { // for each character of arg input

                if (!argOptionals[i] && (Character.isLetter(c) ||
Character.isDigit(c)))

                    valids[i] = true; // required arg must contain at least one
letter or number

            }

            if (argOptionals[i]) // if optional, letter or number requirements
do not apply

                valids[i] = true;

        }

        return valids;

    }

    public void setKeywords(String... names) {

        this.keywords = names;

    }

```

```

protected void setDescription(String description) {
    this.description = description;
}

public void setArgTypes(String... argNames) {
    this.argTypes = argNames;
}

protected void setArgHints(String... argHints) {
    this.argHints = argHints;
}

protected void setArgOptionals(boolean... argOptionals) {
    this.argOptionals = argOptionals;
}

public String[] getKeywords() {
    return keywords;
}

public String getDescription() {
    return description;
}

public String[] getArgTypes() {
    return argTypes;
}

```

```

    public String[] getArgHints() {
        return argHints;
    }

    public boolean[] getArgOptionals() {
        return argOptionals;
    }
}

```

app/src/main/java/org/backend/Commands/ModifyCMD.java

```

Java
package org.backend.commands;

import java.time.LocalDateTime;
import java.util.Arrays;
import org.backend.Interpreter;
import org.backend.Rosetta;
import org.backend.Task;
import org.backend.TaskHandler;

public class ModifyCMD extends Command {
    public ModifyCMD() {
        super();
        setKeywords("edit", "modify", "change", "alter");
        setDescription("Modifies an existing task");
        setArgTypes("Task", "Text", "Duration", "Time");
    }
}

```

```

        setArgHints("Enter the name of the task to modify", "(Optional) Set new
name",

        "(Optional) Set new duration", "(Optional) Set new start time");
        setArgOptionals(false, true, true, true);
    }

    @Override public boolean[] execute(String[] stringArgs) { // edits an
existing task

        boolean[] argValids = verifyArgs(stringArgs);

        if (!Interpreter.boolArrayAllTrue(argValids))

            return argValids;

        Rosetta rosetta = Rosetta.getCurrentInstance();

        String newName = null;

        int newDuration = 0;

        LocalTime newTimeStart = null;

        // get first arg
        Task oldTask = TaskHandler.findTask(stringArgs[0]);

        if (oldTask == null) {

            argValids[0] = false;

            rosetta.displayError("Could not find requested task.");

            return argValids;

        }

        // get second arg

        newName = (!stringArgs[1].isBlank()) ? stringArgs[1] :
oldTask.getName();

```



```

        // get third arg
        newDuration = (!stringArgs[2].isBlank()) ?
Interpreter.parseDuration(stringArgs[2])
: oldTask.getDuration();

        // get fourth arg
        newTimeStart = (!stringArgs[3].isBlank()) ?
Interpreter.parseTime(stringArgs[3])
: oldTask.getTimeStart();

        Task newTask = (newTimeStart != null) ? new Task(newName, newDuration,
newTimeStart)
: new Task(newName, newDuration);

        int statusCode = TaskHandler.replaceTask(oldTask, newTask);

        // error handling
        switch (statusCode) {
            case -2:
                rosetta.displayError("A task with the same name already
exists.");
                argValid[1] = false;
                break;
            case -1:
                rosetta.displayError(
                    "Invalid duration / start time provided. Check your input
values.");
                argValid[2] = false;
                argValid[3] = false;

```

```

        break;

    case 0: // do nothing and be happy

        break;

    case 1:

        rosetta.displayError(

            "There is not enough free time to accomodate the new
duration.");

        argValids[2] = false;

        break;

    case 2:

        rosetta.displayError("New start time will cause tasks to
overlap.");

        argValids[3] = false;

        break;

    default:

        rosetta.displayError(

            "An error occurred. Code: " + statusCode + ". Contact the
developer.");

        Arrays.fill(argValids, false);

        break;

    }

    return argValids;

}

}

```

app/src/main/java/org/backend/Commands/RemoveCMD.java

Java

```
package org.backend.commands;

import org.backend.Interpreter;
import org.backend.Rosetta;
import org.backend.Task;
import org.backend.TaskHandler;

public class RemoveCMD extends Command { // removes an existing task

    public RemoveCMD() {

        super();

        setKeywords("remove", "delete", "trash");

        setDescription("Deletes all occurrences of a task.");

        setArgTypes("Task");

        setArgHints("Enter name of task to delete");

        setArgOptionals(false);

    }

    @Override

    public boolean[] execute(String[] stringArgs) {

        boolean[] argValids = verifyArgs(stringArgs);

        if (!Interpreter.boolArrayAllTrue(argValids))

            return argValids;

        Rosetta rosetta = Rosetta.getCurrentInstance();

        // handle first arg

        Task targetTask = TaskHandler.findTask(stringArgs[0]);

        if (targetTask == null) {
```

```

        argValid[0] = false;

        rosetta.displayError("Could not find requested task.");

        return argValid;
    }

    // execute order 66

    TaskHandler.removeTask(targetTask);

    return argValid;
}
}

```

app/src/main/java/org/openjfx/MainApp.java

```

Java
/*
 * This class was created as instructed through the JavaFX installation guide:
 * https://openjfx.io/openjfx-docs/
 *
 * The code within the start method was mostly sourced from
 * https://jenkov.com/tutorials/javafx/webview.html in order to set up a
 * WebView node.
 *
 * Calls to the class Rosetta were not sourced from anywhere.
 */

package org.openjfx;

import javafx.application.Application;

```

```

import javafx.scene.Scene;

import javafx.scene.layout.VBox;

import javafx.scene.web.WebView;

import javafx.stage.Stage;

import org.backend.Rosetta;


public class MainApp extends Application {

    @Override

    public void start(Stage stage)

        throws Exception { // adapted from
https://jenkov.com/tutorials/javafx/webview.html

        stage.setTitle("pls give level 7");

        stage.setResizable(false);

        WebView wv = new WebView();

        String url =
MainApp.class.getResource("/frontend/index.html").toExternalForm();

        wv.getEngine().load(url); // loads html in resources/frontend


        VBox vBox = new VBox(wv);

        Scene scene = new Scene(vBox, 420, 710); // specifies window dimensions


        wv.setPrefWidth(scene.getWidth());

        wv.setPrefHeight(scene.getHeight());


        Rosetta.initialise(wv);

        Rosetta.startBigBrother(scene); // start listening for keybinds


        stage.setScene(scene);

```

```

        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

app/src/main/resources/frontend/ui-logic/cmd_scripts.js

```

JavaScript
const CMD_VIEW_ID = "scrollable-cmd-view";

const PX_GAP_BETWEEN_CMDS = 5;

let cmdOptions = [];

function deleteCommandView() {
    let cmdView = document.getElementById(CMD_VIEW_ID);
    if (cmdView)
        cmdView.remove();
}

function prepareCommandView() {
    deleteCommandView();

    let cmdView = document.createElement("div");
    cmdView.setAttribute("id", CMD_VIEW_ID);
    document.body.appendChild(cmdView);
    adjustCommandView();
}

```

```

        cmdOptions = [];
    }

    function addCommand(name, description) {

        let cmd = createCommand(name, description);

        let cmdView = document.getElementById(CMD_VIEW_ID);

        let cmdHeight = cmd.offsetHeight;
        let cmdViewHeight = cmdView.offsetHeight;
        let cmdViewNewHeight = cmdHeight + cmdViewHeight;
        cmdView.style.height = cmdViewNewHeight + "px";

        cmd.dataset.index = cmdOptions.push(cmd) - 1;
        cmdView.appendChild(cmd);
        adjustCommandView();
    }

    function createCommand(name, description) {

        let cmd = document.createElement("div");
        cmd.classList.add("cmdUI");

        let cmdName = document.createElement("span");
        cmdName.classList.add("title");
        cmdName.textContent = name;
        cmd.dataset.text = name;

        let cmdDesc = document.createElement("span");
        cmdDesc.classList.add("description");

```

```

    cmdDesc.textContent = description;

    cmd.appendChild(cmdName);
    cmd.appendChild(cmdDesc);

    // cmd.addEventListener("mouseover", (event) => {
    //     selectCommand(event.target);
    // });

    return cmd;
}

function adjustCommandView() {
    let cmdView = document.getElementById(CMD_VIEW_ID);
    let cmdBar = document.getElementById("command-bar");

    // positions the cmdView div to be right above input
    let cmdBarHeight = cmdBar.offsetHeight;
    let cmdViewHeight = cmdView.offsetHeight;

    let cmdViewTopOffset = -(PX_GAP_BETWEEN_CMDS + cmdBarHeight +
cmdViewHeight) + 5;

    cmdView.style.top = cmdViewTopOffset + "px";

    let bodyWidth = window.innerWidth;
    let cmdBarWidth = cmdBar.offsetWidth;
    let cmdViewLeftOffset = (bodyWidth - cmdBarWidth) / 2;
    cmdView.style.left = cmdViewLeftOffset + "px";

```



```

        cmdView.scrollTop = cmdView.scrollHeight; // scrolls to the bottom
    }

function resetCommandView() {
    document.getElementById("args-container").remove();
    let cmdBar = document.getElementById("command-bar");
    cmdBar.innerHTML = cmdBar.dataset.hiddenInnerHTML;
    let cmdInput = document.getElementById("cmd-input");
    cmdInput.value = "";
    cmdInput.focus();

    let argHint = document.getElementById("arg-hint-container");
    if (argHint)
        argHint.remove();

    deleteCommandView();
    rosetta.handleCommandInput(cmdInput.value);
}

function adjustInputWidth() {
    const MIN_CH_WIDTH = 8;
    let cmdInput = document.getElementById("cmd-input");
    let inputLen = cmdInput.value.length;
    if (inputLen > MIN_CH_WIDTH) {
        cmdInput.style.width = inputLen + "ch";
    } else if (inputLen == 0) {
        cmdInput.style.width = "fit-content";
    } else {

```

```

        cmdInput.style.width = MIN_CH_WIDTH + "ch";
    }
}

function handleCommandSubmitKey(key, selectedIndex) {
    let cmdInput = document.getElementById("cmd-input");
    if (document.activeElement.dataset.ident == "argInput") {
        handleArgsNavKey(key); // handle as an arg related action
        return;
    }

    if (document.activeElement !== cmdInput)
        return;

    if (key == "Tab") {
        commandTabIncrement();
    } else if (key == "Escape") {
        deleteCommandView();
    } else { // space or enter
        let cmdBar = document.getElementById("command-bar");
        cmdBar.dataset.oldInputText =
cmdOptions[Number(selectedIndex)].dataset.text;
        cmdBar.dataset.hiddenInnerHTML = cmdBar.innerHTML;
        cmdBar.innerHTML = "";
        deleteCommandView();
        rosetta.populateArgs();
    }
}
}

```

```

function selectCommandAtPos(index) {
    selectCommand(cmdOptions[Number(index)]);
}

function selectCommand(selCmd) {
    cmdOptions.forEach(cmd => {
        cmd.style.background = "#1E1E1E"
    });
    selCmd.style.background = "black";
    rosetta.setSelectedCmdPos(selCmd.dataset.index);
}

function commandTabIncrement() {
    let cmdInput = document.getElementById("cmd-input");
    let currentSelPos = rosetta.getSelectedCmdPos();
    let selCmd = cmdOptions[currentSelPos];

    if (selCmd.dataset.text !== cmdInput.value) {
        selectCommand(selCmd);
        autoCompleteCommand();
        return;
    }

    if (currentSelPos < cmdOptions.length - 1) {
        selCmd = cmdOptions[++currentSelPos];
    } else {
        selCmd = cmdOptions[0];
    }
}

```

```

    }

    selectCommand(selCmd);

    autoCompleteCommand();
}

function autoCompleteCommand() {

    let text = cmdOptions[rosetta.getSelectedCmdPos()].dataset.text;

    let cmdInput = document.getElementById("cmd-input");

    cmdInput.value = text;

}

```

app/src/main/resources/frontend/ui-logic/cmdargs_scripts.js

```

JavaScript
const FOCUS_COLOUR = "rgb(127,74,192)";

const BLUR_COLOUR = "rgb(68, 54, 160)";

const ERROR_COLOUR = "rgb(255,0,0)";


let cmdBar = document.getElementById("command-bar");

let argInputs = [];

let selectedArg = 0;


// handling args


function initArgsContainer() {

    let argsContainer = document.getElementById("args-container");

```

```

    if (argsContainer)
    return;

    argsContainer = document.createElement("div");
    argsContainer.id = "args-container";

    argsContainer.style.left = 10 + "px";
    argsContainer.style.width = cmdBar.clientWidth - 10 + "px";

    cmdBar.appendChild(argsContainer);
}

function createArg(nameContent, index, required) {
    let numInput = false; // this is necessary because only string args can
    be passed by java

    if (nameContent == "Duration")
        numInput = true;

    let argsContainer = document.getElementById("args-container");
    let argBox = document.createElement("div");
    let argName = document.createElement("span");
    let argNameBox = document.createElement("div");
    let argInput = document.createElement("input");

    argBox.classList.add("arg-box");
    argName.classList.add("arg-name");
    argNameBox.classList.add("arg-name-container");
    argInput.classList.add("arg-input");

```

```

        argName.textContent = nameContent + ": ";
        argNameBox.appendChild(argName);
        argBox.appendChild(argNameBox);

        initLengthLimitToInput(argInput, numInput);
        initSelectedListenerToInput(argInput);

        argInputs[Number(index)] = argInput;
        argInput.type = "text";
        argInput.dataset.index = Number(index);
        argInput.dataset.ident = "argInput";
        argInput.dataset.required = required == "true";

        argBox.appendChild(argInput);

        argsContainer.appendChild(argBox);
    }

    function initLengthLimitToInput(input, numInput) {
        let maxLen = 15;

        if (numInput)
            maxLen = 9; // prevent max val abuse

        input.maxLength = maxLen;

        input.addEventListener("input", (_) => {

```

```

        let inputLen = input.value.length;

        input.style.width = inputLen + "ch";

    });
}

function resetArgInputs() {
    argInputs = [];
}

function handleArgsNavKey(key) {
    if (key == "Enter") {
        selectedArg = 0;
        inputVals = [];
        argInputs.forEach(input => {
            inputVals.push(input.value.trim());
        });
        rosetta.executeCommand(inputVals);
    } else if (key == "Tab") {
        incrementSelectedArg();
    } else if (key == "Escape") { // restore old input state
        selectedArg = 0;
        inputVals = [];
        argInputs = [];
        resetCommandView();
        let cmdInput = document.getElementById("cmd-input");
        cmdInput.value = cmdBar.dataset.oldInputText;
        rosetta.handleCommandInput(cmdInput.value);
    }
}

```

```

}

function initSelectedListenerToInput(input) {
    input.addEventListener("blur", (event) => {
        let selectedArgBox = event.target.parentElement;
        selectedArgBox.style.borderColor = BLUR_COLOUR;

        let argHint = document.getElementById("arg-hint-container");
        argHint.remove();
    });

    input.addEventListener("focus", (event) => showArgumentHint(event));
    input.addEventListener("keydown", (event) => showArgumentHint(event));

    function showArgumentHint(event) {
        let selectedArgBox = event.target.parentElement;
        selectedArgBox.style.borderColor = FOCUS_COLOUR;
        rosetta.requestArgHint(event.target.dataset.index);
        if (input.textContent.length == 1)
            input.textContent = input.textContent.trim();
    }
}

function incrementSelectedArg() {
    if (selectedArg < argInputs.length - 1)
        selectedArg++;
    else
        selectedArg = 0;
}

```



```

        updateSelectedArg();
    }

    function updateSelectedArg() {
        argInputs.forEach(input => {
            input.value = input.value.trimStart();
            input.blur();
        });
        let selectedInput = argInputs[selectedArg];
        selectedInput.focus();
        selectedInput.value = selectedInput.value.trim();
    }

    function setArgAsInvalid(strIndex) {
        let index = Number(strIndex);
        argInputs[index].parentElement.style.borderColor = ERROR_COLOUR;
    }

    function setArgAsNormal(strIndex) {
        let index = Number(strIndex);
        argInputs[index].parentElement.style.borderColor = BLUR_COLOUR;
    }

    // handling arg hints
    const ARG_HINT_CONTAINER_ID = "arg-hint-container";
    const ARG_HINT_TITLE_ID = "arg-hint-title";
    const ARG_HINT_BODY_ID = "arg-hint-body";

```

```

function showArgHint(argTitle, argBody) {

    let argHint = document.getElementById(ARG_HINT_CONTAINER_ID);

    if (!argHint) {

        argHint = document.createElement("div");

        argHint.id = ARG_HINT_CONTAINER_ID

        let leftOffset = cmdBar.getBoundingClientRect().x;

        let topOffset = cmdBar.getBoundingClientRect().y -
argHint.getBoundingClientRect().height - 25;

        argHint.style.left = leftOffset + "px";

        argHint.style.top = topOffset + "px";

        argHint.style.width = cmdBar.getBoundingClientRect().width - 5 + "px";

        document.body.appendChild(argHint);

    }

    argHint.innerHTML = "";

    let argHintTitle = document.createElement("span");

    argHintTitle.id = ARG_HINT_TITLE_ID;

    argHintTitle.textContent = argTitle;

    argHint.appendChild(argHintTitle);

    let argHintBody = document.createElement("span");

    argHintBody.id = ARG_HINT_BODY_ID;

    argHintBody.textContent = argBody;

    argHint.appendChild(argHintBody);

```

```

}

function showError(errorMsg) {
    showArgHint("", "");

    let title = document.getElementById(ARG_HINT_TITLE_ID);
    let body = document.getElementById(ARG_HINT_BODY_ID);

    title.textContent = "error"
    body.textContent = errorMsg;

    body.style.color = "red";
}

```

app/src/main/resources/frontend/ui-logic/task_scripts.js

JavaScript

```

function addTask(titleText, timeStart, timeEnd, statusText) {

    let taskUI = document.createElement("div");
    taskUI.classList.add("taskUI");

    let title = document.createElement("p");
    title.classList.add("title");
    title.textContent = titleText;
    taskUI.appendChild(title);

    let time = document.createElement("p");
    time.classList.add("time");
    time.textContent = timeStart + " - " + timeEnd;
}

```

```

taskUI.appendChild(time);

let status = document.createElement("p");
status.classList.add("fixedStatus");
status.textContent = statusText;
taskUI.appendChild(status);

document.getElementById("scrollable-task-view").appendChild(taskUI);
}

function clearTaskList() {
    let taskView = document.getElementById("scrollable-task-view");
    taskView.innerHTML = ""; // gets rid of children
}

```

app/src/main/resources/frontend/ui-styles/cmd_styles.css

```

CSS
#scrollable-cmd-view {
    z-index: 2;
    position: relative;
    width: 75%;
    height: 5px;
    border: 1.5px solid rgb(55, 55, 80);
    background-color: rgb(30, 30, 30);
    border-radius: 10px;
    overflow-y: auto;
    padding: 10px;
}

```

```

        display: flex;

        flex-direction: column;

        gap: 8px;

        box-shadow: 0, 0, 10px, black;
    }

    #scrollable-cmd-view::-webkit-scrollbar {

        display: none;
    }

    .cmdUI {

        z-index: 3;

        width: 95%;

        height: 40px;

        position: relative;

        background-color: rgb(30, 30, 30);

        border-radius: 10px;

        display: flex;

        padding: 5px;

        margin-left: auto;

        margin-right: auto;
    }

    .cmdUI>.title {

        z-index: 4;

        font-size: 12px;

        color: white;

        position: absolute;

```

```

    left: 10px;

    top: 50%;

    transform: translateY(-50%);
    transform: translate(0, -50%);
}

.cmdUI>.description {
    z-index: 4;

    font-size: 10px;

    color: rgb(160, 160, 160);

    right: 10px;

    position: absolute;

    top: 50%;

    transform: translateY(-50%);
    transform: translate(0, -50%);
}

```

app/src/main/resources/frontend/ui-styles/cmdargs_styles.css

```

CSS
#args-container {
    z-index: 5;

    overflow-y: hidden;

    overflow-x: auto;

    scroll-behavior: smooth;

    height: 100%;

    background-color: transparent;

    position: absolute;

```

```

    display: flex;

    gap: 5px;

    padding-left: 2px;

    padding-right: 2px;
}

/*adapted from chatgpt*/

#args-container::-webkit-scrollbar {

    display: none;
}

.arg-box {

    z-index: 6;

    will-change: contents;

    width: fit-content;

    height: 80%;

    position: relative;

    display: flex;

    background-color: transparent;

    border: 2px solid rgb(68, 54, 160);

    border-radius: 2px;

    gap: 5px;

    top: 50%;

    transform: translateY(-50%);
}

.arg-name-container {

    z-index: 7;

```

```
padding-left: 4px;

padding-right: 4px;

background-color: rgb(53, 29, 77);

display: flex;

position: relative;

height: 100%;

width: fit-content;

left: 0;

top: 50%;

transform: translateY(-50%);

}

.arg-name {

    z-index: inherit;

    display: flex;

    position: relative;

    height: fit-content;

    text-align: left;

    font-family: 'Montserrat', system-ui;

    font-style: normal;

    font-weight: 700;

    color: rgb(205, 192, 255);

    font-size: 10px;

    left: 0;

    top: 50%;

    transform: translateY(-50%);

}
```



```
.arg-input {  
    z-index: 7;  
    width: 20px;  
    position: relative;  
    height: 98%;  
    background: none;  
    color: white;  
    font-family: 'Courier New', Courier, monospace;  
    font-weight: 700;  
    font-size: 12px;  
    text-align: left;  
    border: none;  
    outline: none;  
}
```

```
#arg-hint-container {  
    position: absolute;  
    display: flex;  
    width: 85%;  
    height: 16px;  
    padding: 3px;  
    background-color: rgb(16, 15, 90);  
}
```

```
#arg-hint-title {  
    position: absolute;  
    display: flex;  
    width: fit-content;
```

```
height: fit-content;

right: 11px;

font-family: 'Montserrat', system-ui;

font-size: 11px;

font-style: normal;

font-weight: bold;

text-align: right;

color: rgb(202, 202, 202);

top: 50%;

transform: translateY(-50%);

}

#arg-hint-body {

    position: absolute;

    display: flex;

    width: fit-content;

    height: fit-content;

    left: 11px;

    font-family: 'Montserrat', system-ui;

    font-size: 10px;

    font-style: normal;

    font-weight: 400;

    text-align: left;

    color: whitesmoke;

    top: 50%;

    transform: translateY(-50%);

}
```

app/src/main/resources/frontend/ui-styles/const_styles.css

```
CSS
body {

    height: 100vh;

    margin: 0px;

    overflow: hidden;

    background-color: rgb(10, 8, 22);

    font-family: 'Montserrat', system-ui;

}


#scrollable-task-view {

    width: 85%;

    height: 80%;

    border: 1.5px solid rgb(55, 55, 80);

    background-color: rgb(2, 9, 34);

    border-radius: 10px;

    overflow-y: auto;

    padding: 10px;

    display: flex;

    flex-direction: column;

    gap: 10px;

    margin-left: auto;

    margin-right: auto;

    margin-top: 20px;

}


.taskUI {

    width: 96%;

    height: 60px;
```

```
    position: relative;

    background-color: rgba(0, 49, 102, 0.372);

    border: 1px solid rgba(0, 71, 148);

    border-radius: 10px;

    display: block;

    padding-top: 5px;

    padding-left: 10px;

    margin-left: auto;

    margin-right: auto;

}
```

```
.taskUI>.title {

    z-index: 2;

    height: 7px;

    position: relative;

    margin-top: 2px;

    width: fit-content;

    font-family: 'Montserrat', system-ui;

    font-size: 13px;

    font-style: normal;

    font-weight: 700;

    color: white;

}
```

```
.taskUI>.time {

    z-index: 2;

    height: 5px;

    position: relative;
```

```
margin-top: 11px;

width: fit-content;

font-family: 'Montserrat', system-ui;

font-size: 10px;

font-style: normal;

font-weight: 500;

color: rgb(200, 200, 200);

}
```

```
.taskUI>.fixedStatus {

  z-index: 2;

  height: 5px;

  position: relative;

  margin-top: 13px;

  width: fit-content;

  font-family: 'Montserrat', system-ui;

  font-size: 8px;

  font-style: italic;

  font-weight: 400;

  color: rgb(200, 200, 200);

}
```

```
#command-bar {

  width: 85%;

  height: 38px;

  background-color: black;

  border: 1.5px solid rgb(45, 45, 70);

  border-radius: 10px;
```

```
padding-left: 10px;

padding-right: 10px;

margin-left: auto;

margin-right: auto;

margin-top: 40px;

position: relative;

}


#cmd-input {

    position: absolute;

    outline: none;

    font-size: 11px;

    background-color: transparent;

    color: white;

    height: 100%;

    width: fit-content;

    border: none;

    top: 50%;

    transform: translateY(-50%);

    transform: translate(0, -50%);

    font-family: 'Montserrat', system-ui;

}


#cmd-input::placeholder {

    color: gray;

    font-size: 12px;

    font-weight: 600;

    font-family: 'Courier New', Courier, monospace;
```

```

}

#made-with {
    color: gray;
    font-style: normal;
    font-family: 'Montserrat', system-ui;
    font-size: 8px;
    bottom: 10px;
    position: absolute;
    left: 50%;
    transform: translateX(-50%);
}

```

app/src/main/resources/frontend/index.html

```

HTML
<!DOCTYPE html>

<head>
    <title>Home</title>
    <link rel="stylesheet" href="ui-styles/const_styles.css">
    <link rel="stylesheet" href="ui-styles/cmd_styles.css">
    <link rel="stylesheet" href="ui-styles/cmdargs_styles.css">
</head>

<body>
    <div id="scrollable-task-view"></div>
    <div id="command-bar">

```

```
<input id="cmd-input" placeholder="Enter a command" maxlength="15"
      oninput="rosetta.handleCommandInput(this.value)"
onfocus="rosetta.handleCommandInput(this.value)"
      onblur="rosetta.handleCommandunfocus()">
</div>
<script src="ui-logic/task_scripts.js"></script>
<script src="ui-logic/cmd_scripts.js"></script>
<script src="ui-logic/cmdargs_scripts.js"></script>
</body>

</html>
```